

Python and LaTeX Module 4

Syllabus: Files and user I/O User input , Reading files , Writing to files , Directories , Interactive programming , Pickling , Exceptions

Files and User I/O in Python

Python provides functions to **interact with users and handle files**.

1. User Input

- Get input from the user using `input()`.
- Input is always **string**, convert if needed.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name, "you are", age, "years old.")
```

2. File Handling

a) Reading Files

```
f = open("example.txt", "r") # Open file in read mode
content = f.read()          # Read entire file
print(content)
f.close()
```

b) Writing Files

```
f = open("example.txt", "w") # Write mode (overwrites file)
f.write("Hello, World!")
f.close()
```

c) Using with Statement (*Recommended*)

```
with open("example.txt", "r") as f:
    content = f.read()
    print(content)
```

- Automatically closes the file.

3. Directories

- Use the os module to manage directories and files.

```
import os
print(os.getcwd())    # Current directory
os.mkdir("new_folder") # Create folder
os.listdir()          # List files/folders
os.rename("old.txt", "new.txt") # Rename file
```

4. Pickling

- Save and load Python objects using pickle.

```
import pickle
data = {"name": "Athulya", "age": 21}
```

Save object

```
with open("data.pkl", "wb") as f:
    pickle.dump(data, f)
```

Load object

```
with open("data.pkl", "rb") as f:
    loaded_data = pickle.load(f)
print(loaded_data)
```

5. Exceptions (Error Handling)

- Handle runtime errors using try-except.

try:

```
x = int(input("Enter a number: "))
print(10 / x)
```

except ValueError:

```
print("Enter a valid number!")  
except ZeroDivisionError:  
    print("Cannot divide by zero!")  
finally:  
    print("Execution finished")
```

✓ Key Points

1. input() → user input
2. open() → read/write files
3. with → safely handle files
4. os module → directories and files
5. pickle → save/load objects
6. try-except → handle exceptions

User Input in Python

User input allows a program to interact with the user by taking data from the keyboard. Python provides the built-in input() function for this purpose.

1. Basic Input

```
name = input("Enter your name: ")  
print("Hello", name)
```

- Displays a **prompt** and waits for the user to enter a value.
- Input is **always returned as a string**.

2. Type Conversion

Since input() returns a string, it may need to be converted to other types:

```
age = int(input("Enter your age: "))    # Convert to integer  
height = float(input("Enter your height: ")) # Convert to float
```

3. Multiple Inputs

You can take multiple inputs at once and split them:

```
x, y = map(int, input("Enter two numbers: ").split())  
print("Sum:", x + y)
```

- `split()` separates input by spaces.
- `map()` converts each input to the required type.

4. Example of Interactive Program

```
name = input("Enter your name: ")  
age = int(input("Enter your age: "))  
print(f'Hello {name}, next year you will be {age+1} years old!')
```

✅ Key Points:

1. `input()` is used for user input.
2. Input is always a string; convert if needed.
3. Can handle single or multiple inputs.
4. Essential for interactive programs.

Reading Files in Python

Python allows programs to read data from files stored on the computer. Files can contain text, numbers, or other data. The built-in `open()` function is used to open a file in read mode ("r").

1. Reading Entire File

```
f = open("example.txt", "r") # Open file in read mode  
content = f.read()          # Read entire file  
print(content)  
f.close()                   # Close the file
```

- "r" mode is used to **read files**.
- Always **close the file** to free resources.

2. Reading Line by Line

```
f = open("example.txt", "r")  
for line in f:  
    print(line.strip())    # strip() removes newline characters  
f.close()
```

3. Using readline()

```
f = open("example.txt", "r")  
line1 = f.readline()    # Reads first line  
line2 = f.readline()    # Reads second line  
print(line1, line2)  
f.close()
```

4. Using readlines()

```
f = open("example.txt", "r")  
lines = f.readlines()    # Returns a list of all lines  
print(lines)  
f.close()
```

5. Using with Statement (Recommended)

```
with open("example.txt", "r") as f:  
    content = f.read()  
    print(content)  
    • Automatically closes the file, even if an error occurs.
```

Key Points:

1. Use open() with "r" mode to **read files**.
2. Methods: read(), readline(), readlines().
3. **with statement** is best practice for file handling.
4. Always **close files** if not using with.

Writing to Files in Python

Python allows programs to write data to files stored on the computer. Files can be opened in write ("w") or append ("a") mode using the open() function.

1. Writing to a File

```
f = open("example.txt", "w") # Write mode (creates file or overwrites)
```

```
f.write("Hello, World!\n")
```

```
f.write("Python file handling")
```

```
f.close()
```

- "w" mode **overwrites** the file if it already exists.
- "a" mode **appends** data at the end of the file without removing existing content.

2. Writing Multiple Lines

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
```

```
f = open("example.txt", "w")
```

```
f.writelines(lines) # Write a list of lines
```

```
f.close()
```

3. Using with Statement (Recommended)

with open("example.txt", "w") as f:

```
    f.write("Hello!\n")
```

```
    f.write("Writing to files in Python.")
```

- Automatically **closes the file** after writing, even if errors occur.

4. Key Points

1. Use open() with "w" or "a" mode to write files.
2. "w" → overwrites file, "a" → appends data.
3. Methods: write() for single string, writelines() for multiple lines.
4. Using with is **best practice** to handle files safely.

Directories in Python

Python provides the **os module** to work with directories and manage files. It allows programs to **create, delete, list, and navigate directories**.

1. Current Directory

```
import os  
  
print(os.getcwd()) # Get current working directory
```

2. Creating Directories

```
os.mkdir("new_folder") # Create a single folder  
  
os.makedirs("folder1/folder2") # Create nested folders
```

3. Listing Files and Folders

```
print(os.listdir()) # List all files and directories in current folder
```

4. Changing Directory

```
os.chdir("new_folder") # Change current working directory  
  
print(os.getcwd())
```

5. Removing Directories

```
os.rmdir("new_folder") # Remove empty folder  
  
os.removedirs("folder1/folder2") # Remove nested empty folders
```

6. Renaming Files or Directories

```
os.rename("old.txt", "new.txt") # Rename a file or folder
```

Key Points

1. os module manages **directories and paths**.
2. mkdir() creates a folder, rmdir() removes it.
3. os.listdir() lists contents.
4. os.chdir() changes the current directory.
5. os.rename() renames files or directories.

Interactive Programming in Python

Interactive programming allows programs to communicate with users during execution. The program can take input, process it, and display output immediately, making it more dynamic and user-friendly.

1. Taking Input

Use the `input()` function to get data from the user:

```
name = input("Enter your name: ")  
  
print("Hello,", name)
```

- Input is always a string; convert to other types if needed:

```
age = int(input("Enter your age: "))
```

2. Processing Input

You can perform calculations or operations based on user input:

```
num = int(input("Enter a number: "))  
  
print("Square:", num**2)
```

3. Using Loops for Interaction

Programs can **continuously interact** with the user until a condition is met:

while True:

```
    command = input("Enter command (exit to quit): ")  
  
    if command.lower() == "exit":  
        break  
  
    print("You entered:", command)
```

4. Benefits

1. Programs become **dynamic** and **user-friendly**.
2. Can **validate input** and respond to user commands in real-time.
3. Essential for **command-line tools, games, calculators, and quizzes**.

✅ Key Points:

- Use `input()` for user input.
- Convert input to the required type if necessary.
- Combine with **loops and conditions** for interactive behavior.

Pickling in Python

Pickling is the process of converting a Python object into a byte stream so that it can be saved to a file and retrieved later. It is useful for storing complex data structures like lists, dictionaries, and custom objects.

1. Import Pickle Module

```
import pickle
```

2. Pickle (Save) an Object

```
data = {"name": "Athulya", "age": 21}
```

```
with open("data.pkl", "wb") as f: # 'wb' → write binary
```

```
    pickle.dump(data, f)
```

- `dump()` saves the object to a file.
- Binary mode (`wb`) is required.

3. Unpickle (Load) an Object

```
with open("data.pkl", "rb") as f: # 'rb' → read binary
```

```
    loaded_data = pickle.load(f)
```

```
print(loaded_data) # Output: {'name': 'Athulya', 'age': 21}
```

- `load()` reads the object back from the file.

4. Key Points

1. Pickling **saves Python objects** for later use.
2. Uses **binary files**, so always open with `'wb'` or `'rb'`.
3. Can store **lists, dictionaries, custom classes, etc.**
4. Useful for **persistent data storage**, caching, or transferring data between programs.

Exceptions in Python

Exceptions are errors that occur during program execution. Python provides a way to handle these errors gracefully using try, except, else, and finally blocks. Handling exceptions prevents the program from crashing.

1. Basic Exception Handling

try:

```
x = int(input("Enter a number: "))  
print(10 / x)
```

except ValueError:

```
print("Invalid input! Please enter a number.")
```

except ZeroDivisionError:

```
print("Cannot divide by zero!")
```

- **try:** Block of code to test for errors.
- **except:** Executes if an error occurs.

2. Using else

- Runs if **no exception occurs**.

try:

```
num = int(input("Enter a number: "))
```

except ValueError:

```
print("Invalid input!")
```

else:

```
print("You entered:", num)
```

3. Using finally

- Executes **always**, whether an exception occurs or not.

try:

```
f = open("file.txt", "r")
```

except FileNotFoundError:

```
    print("File not found!")
finally:
    print("Execution finished.")
```

4. Raising Exceptions

- You can **raise your own exceptions** using raise.

```
age = int(input("Enter age: "))
if age < 0:
    raise ValueError("Age cannot be negative!")
```

Key Points

1. Exceptions prevent the program from crashing.
2. Use try-except to handle errors.
3. else runs if no errors occur, finally runs always.
4. Common exceptions: ValueError, ZeroDivisionError, FileNotFoundError, etc.
5. You can raise custom exceptions with raise.